COMPILER DESIGN · PROJECT REVIEW 2

Real-Time Parallel Log / Event Triage Engine

A regex → NFA → DFA compiler and a lock-free multi-core pipeline for classifying streaming logs into authentication failures, security threats, errors, performance warnings and personal data

Team 1
24BCI0099 — Adithya Santhosh P
24BCT0055 — Anvesha Tripathi

Review 2 report · 14 October 2026

Source code: github.com/AdithyaSanthosh-P/Real-Time-Parallel-Log-Event-Triage-Engine

Contents

1. Abstract

2. Summary of progress

3. System architecture

4. Implementation steps

5. Module-wise split and work done

6. Verification and results so far

7. Remaining work

References

1. Abstract

Servers and network services emit thousands of log lines per second, and the few that matter — repeated failed logins, injection attempts, crashes, slow queries, leaked card or Aadhaar numbers — must be found as they arrive. This project builds a real-time triage engine that classifies every incoming log line into **AUTH_FAILURE**, **SECURITY_THREAT**, **ERROR**, **PERF_WARNING** and **PII**, with a severity and a confidence score, and routes urgent lines to an alert stream.

The core is a compiler built for the purpose: the 26 detection rules, written as POSIX-style regular expressions, are lexed and parsed by a recursive-descent parser, turned into a Thompson NFA and compiled by subset construction into one combined DFA whose states carry a bit mask of the rules matched. Every line is therefore checked against all rules in a single linear pass, with no backtracking, and the read-only DFA tables are shared by all worker threads without locks. Around this matcher the engine adds checksum validators (Luhn, Verhoeff) to suppress false positives, a noisy-OR confidence model with multi-label output, masking of personal data, a correlator that detects attacks spread over many lines (brute force, password spraying), and a multi-threaded pipeline connected by lock-free single-producer/single-consumer rings.

The project extends the base paper on parallel regex text classification. We first reproduced that paper's design and measured its claims: spawning one process per pattern per input costs far more than the matching it parallelises, while parallelism over input lines — the approach taken here — scales. At this review the compiler, the classification logic, the correlator and the parallel runtime are complete and verified; benchmarking and accuracy evaluation follow.

2. Summary of progress

Work follows an ordered implementation plan in which every task has an explicit, testable completion criterion. A correct single-threaded pipeline was built first on the C library's regex engine; our compiler then replaced the matcher and was checked against it; classification logic was added next, and only then was the pipeline parallelised, each stage being compared with the output of the simpler one before it.

Phase	Scope	Status
0	Repository, build system (warning-free C11, sanitizer builds), test runner, sample logs	Complete
1	Single-threaded baseline: rule loader, normaliser, reader, POSIX regex engine, output	Complete
2	Regex compiler: lexer, parser, Thompson NFA, subset-construction DFA, DFA engine, differential tester, Graphviz export	Complete
R	Faithful reproduction and benchmark of the base paper (6 claims evaluated)	Complete (optional R5 open)
3	Validators, entity extraction, confidence model, multi-label, PII masking	Complete
4	Parallel runtime: lock-free rings, job pool, workers, dispatcher, stats, shutdown	Complete
5	Correlator: brute force, password spray, fail-then-success, error storm	Complete
6	Benchmark tooling: fork engine, --bench mode, log generator, rule sets, runner	Next
7	Evaluation: full datasets, hand-labelled accuracy (P/R/F1, Cohen's κ), experiments	Planned

Key facts at this review

Measure	Value
Planned tasks completed	32 of 41 required tasks (78 %)
Optional and stretch tasks	not started (1 optional, 11 stretch)
Engine source (src/, C11)	7,173 lines in 29 modules
Tests (unit, end-to-end, scripts)	5,076 lines; 19 unit-test programs
Base-paper reproduction (tools/basepaper/)	2,099 lines
Detection rules	26 rules, 5 categories

Highlights

- The compiled DFA agrees with the C library's *regexec* on every rule for every sample log line and 100,000 generated strings (0 mismatches).
- The C engine and an independent Python re-implementation agree on primary label, labels, severity, confidence and escalation flags for every line of six sample logs (10,000+ lines).
- Output with 1, 2, 4 and 8 worker threads is identical to the single-threaded output (72 comparison runs); all concurrent code runs clean under ThreadSanitizer.
- On the public Loghub OpenSSH sample the correlator raises 34 brute-force and 5 password-spray escalations.
- Base paper: its fork-per-pattern design measured 12–13× slower than a sequential loop that recompiles the patterns; data-parallel variants scale (results in `results/basepaper/`).

3. System architecture

The system has two parts. At start-up the rule set is compiled into DFA tables (Figure 1). At run time a pipeline of threads streams log lines through matching, classification, correlation and output (Figure 2). Stages exchange *batches* of lines (256 by default) through bounded single-producer/single-consumer rings; only the workers scale with `-w N`.

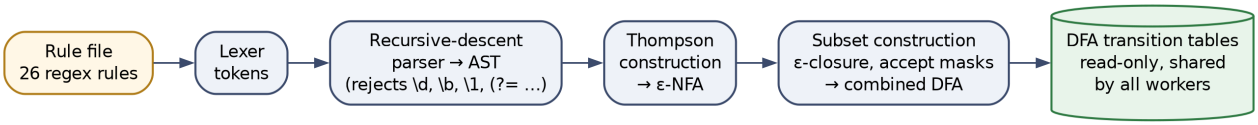


Figure 1. Rule compilation (once at start-up): the front end of a compiler, from regular-expression source to DFA transition tables.

Compilation. A lexer and a recursive-descent parser (one function per grammar non-terminal) build an abstract syntax tree; constructs outside the regular POSIX-ERE subset (`\d`, `\b`, backreferences, lookaround, lazy quantifiers) are rejected with their position. Thompson's construction gives one ε-NFA for all rules, each accept state tagged with its rule. Subset construction (ε-closure over NFA-state sets) yields a combined DFA in which every state stores a 64-bit accept mask; to match anywhere in a line the start set is re-injected at every step. If the DFA would exceed the state cap, the rules are split into groups with one DFA each (the 26 default rules: 1,762 NFA states, 2 DFA groups, 7,593 DFA states).

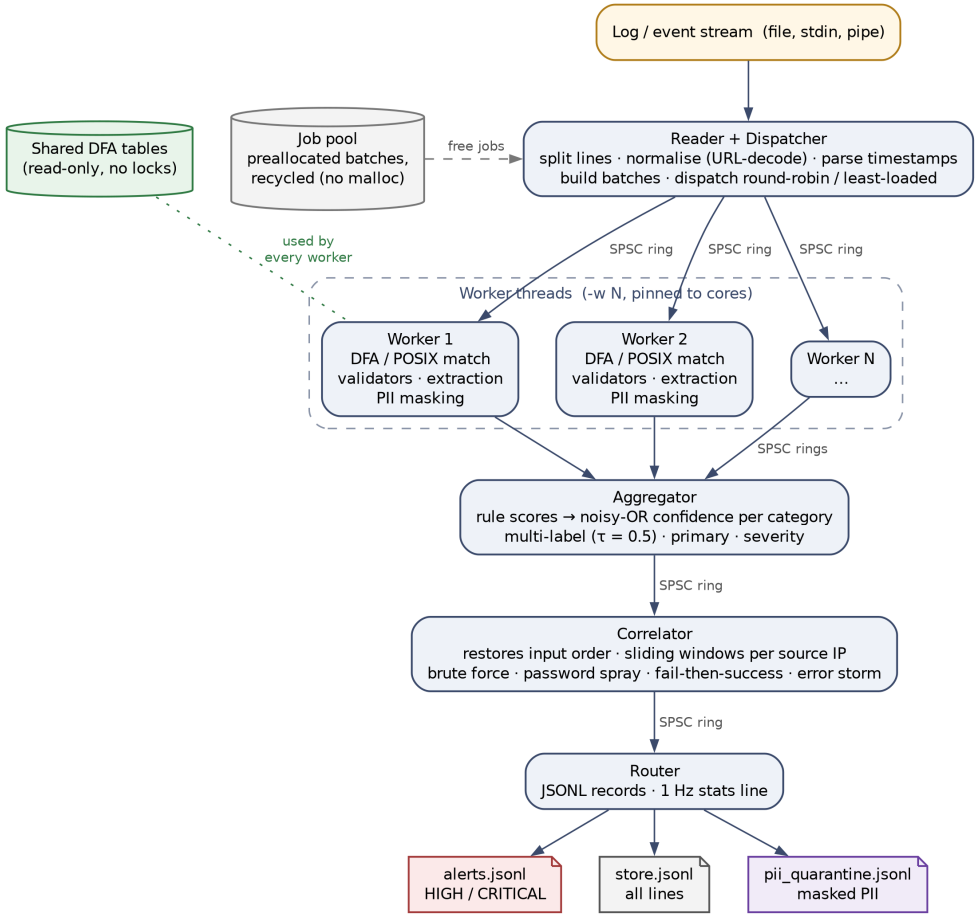


Figure 2. Run-time architecture: one thread per stage, lock-free SPSC rings between stages, worker threads sharing the read-only DFA tables.

Run time. The reader splits the input into lines, decodes URL escapes for matching, parses timestamps and fills preallocated batches, which the dispatcher hands to workers (round-robin or least-loaded). Each

worker matches lines with the DFA (or, as a baseline, one *regexec* per rule) and, for matched lines only, runs validators, extracts the source IP and user name, and masks personal data. The aggregator turns rule matches into per-category confidence and labels; the correlator restores input order and applies sliding time windows per source IP; the router writes JSON-lines records to three sinks and prints a status line every second. Finished batches return to the reader through a free ring, so the steady state allocates no memory. Full rings block the stage before them (back-pressure, no data loss), or drop and count batches with `--drop`.

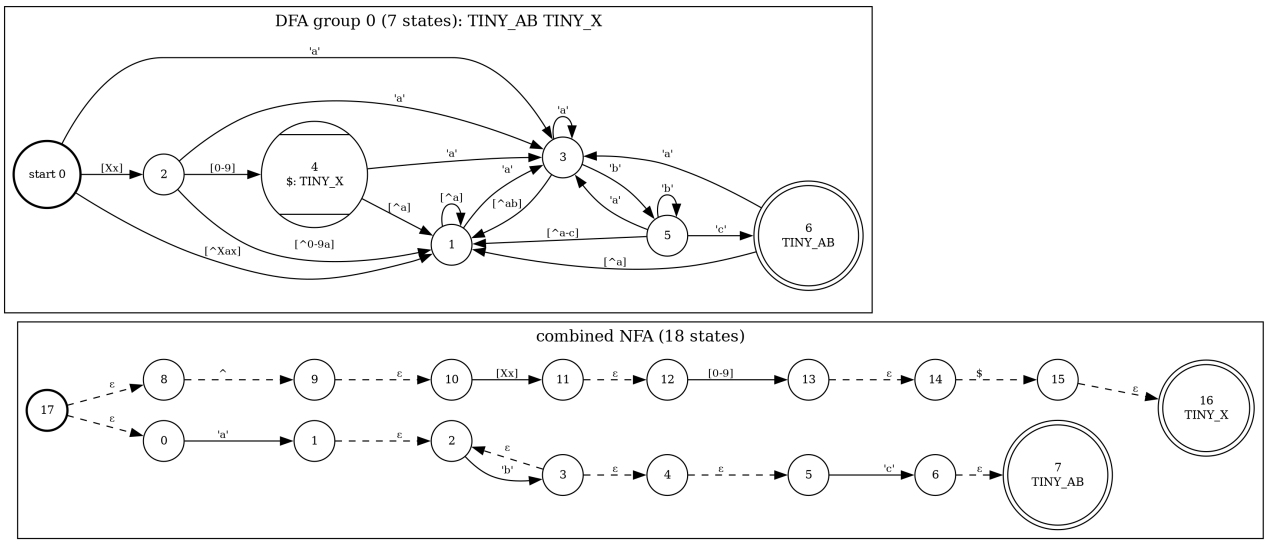


Figure 3. Output of the engine's own `--dump-dot` for a two-rule file (`ab+c` and `^x[0-9]$`): the Thompson NFA (bottom, dashed ε-edges) and the 7-state combined DFA (top) with collapsed byte-range labels.

4. Implementation steps

The steps below are in the order they were carried out. Each was finished only when its verification check and the full test suite passed.

#	Step	What was built	How it was verified
1	Baseline pipeline	TSV rule loader with validation; normaliser (URL-decoding, control characters); chunked reader with batching and timestamp parsing; POSIX engine; JSONL router; command-line interface	Hand-annotated corpus (every rule has a positive line and near-misses); expected-output file
2	Regex front end	Lexer and recursive-descent parser to an AST with 256-bit character sets; precise errors for unsupported syntax	97 parser cases (valid and invalid); all 26 rules parse
3	Thompson NFA	One construction per AST node, anchors as assertion edges, combined NFA; set-of-states simulator	NFA result = regexec on every rule and test string
4	Subset construction	ε-closure, hashed state sets, accept masks, byte classes, state cap with rule-group splitting	DFA result = NFA simulator
5	DFA engine + difftest	Table-driven matcher behind `--engine dfa`; differential test tool	0 mismatches on all corpus lines and 100,000 random strings
6	Base-paper reproduction	Paper's fork-per-pattern program, five comparison variants, labelled set of 1,250 inputs, benchmark runner, report	5 runs per data point; verdict per claim
7	Classification	Luhn, Verhoeff and PAN validators; span and entity extraction; context keywords; noisy-OR confidence, τ threshold, multi-label, primary and severity	Property tests; independent Python reference classifier
8	PII masking	Format-preserving masks; quarantine sink	End-to-end test searches all output for every PII value
9	Parallel runtime	SPSC rings, job pool, workers with CPU pinning, dispatcher (rr/least, batch delay, drop), aggregator and router threads, stats, error shutdown	10-million-item ring stress test; ThreadSanitizer; parallel = sequential on all logs
10	Correlator	Per-IP hash map with failure rings; brute force, password spray, fail-then-success, error storm; in-order release of batches	Crafted sequences; agreement with Python reference on real OpenSSH log
11	Demo and tooling	Scripted review demo, Graphviz export, comparison and allocation checks	Runs end to end on sample logs

5. Module-wise split and work done

The engine is divided into modules with one header per source file and no shared mutable state apart from the parsed configuration. Line counts include headers.

Module	Source files	Responsibility	Work done	Lines
CLI and driver	main, cli	Command-line options, help, exit codes	All options of the specification parsed and validated; engine, workers, batching, dispatch, pinning, drop, τ, state cap, DOT export implemented	321
Rule loader	rules	Load and validate the rule file	26 rules; column, category, severity, validator and duplicate checks with file:line errors	417
Input	reader, normalize, timestamp, batch	Read stream, split lines, normalise, timestamps, batches	CRLF, long lines, empty input; URL-decoding; syslog and ISO-8601 time; batch-delay flush for pipes	683
Regex compiler	regex_lex, regex_parse, ast, nfa, dfa, dot	Rules to DFA tables; diagrams	Recursive-descent parser with error positions; Thompson NFA and simulator; subset construction with accept masks and group splitting; DOT export	2,305
Matching engines	engine, engine_dfa, engine_posix	Uniform interface; per-thread instances	DFA engine on shared tables; POSIX baseline with per-thread compiled regexes	312
Validation and extraction	validators, extract	Checksums, spans, IP and user, context	Luhn, Verhoeff, PAN; capture spans; IPv4 and user-name extraction; context keywords; subsumption	500
Classification	aggregate	Confidence, labels, severity	Rule scores, noisy-OR per category, τ = 0.5, multi-label, primary by severity and tie order, escalation hook	213
PII masking	mask	Mask personal data in output	Card, Aadhaar, PAN, secrets, e-mail, phone; only validator-passed values	160
Correlator	correlate	Multi-line attack detection	Per-IP windows on log time; brute force, password spray, fail-then-success, error storm; once-per-window emission	327
Output	route, json	JSONL records and sinks	alerts, store and PII-quarantine sinks; strict JSON escaping including invalid UTF-8	501
Parallel runtime	spsc, pool, worker, dispatch, pipeline, stats	Threads, queues, scheduling, shutdown	Lock-free rings (release/acquire, cache-line separated); preallocated job pool; pinned workers; rr and least-loaded dispatch; back-pressure and drop; ordered correlation; 1 Hz statistics; stop-on-error	1,354
Base-paper reproduction	tools/basepaper/	Rebuild and measure the base paper	Faithful fork program, five variants, labelled data, runner, figures, verdict report	2,099
Verification tools	tests/, tools/	Correctness checks	19 unit-test programs, differential tester, Python reference classifier, end-to-end and run-equivalence scripts, demo	6,613

Features not yet implemented

- **Fork engine inside the triage binary** (`--engine fork`): the base paper's mechanism applied to the log rules, as a baseline for experiments.
- **Benchmark mode** (`--bench`): preloaded input, throughput and latency percentiles as JSON.
- **Synthetic log generator** with exact ground truth and controllable attack and PII rates.

6. Verification and results so far

Check	What it establishes	Result
Unit tests (make test)	Each module in isolation, incl. 10M-item ring stress test	19 programs pass
Differential test (make difftest)	DFA = C library regexec per rule	0 mismatches
End-to-end (make e2e)	Labels, severity, flags vs. Python reference; no PII in output; alerts = HIGH/CRITICAL; clean exit on write error	pass
Run equivalence (make compare)	-w 1, 2, 4, 8 identical, both engines, all sample logs	72 runs equal
Sanitizers	AddressSanitizer and ThreadSanitizer on tests and end-to-end runs	no reports
Allocation check	No memory growth with input size (100k vs 500k lines)	constant

Base-paper reproduction

The base paper was rebuilt as described — one *fork()* per pattern per input, with *regcomp* and *regfree* in every child — and compared with sequential, thread-based and DFA variants on the same five patterns (5 runs per point, Intel i7-13650HX). Selected results; the complete tables and figures are in `results/basepaper/SUMMARY.md`.

Base-paper claim	Paper	Our measurement	Verdict
Parallel matching speeds up classification	2.4×	fork design 0.08× of a sequential recompiling loop; data-parallel threads 3.1×, combined DFA 25× (5 threads)	Not reproduced for the paper's design
Near-linear scaling to 6 cores	4.87× at 6	fork 1.65× at 6; threads over inputs 4.83× at 6	Not reproduced for the paper's design
8.9 ms sequential vs 3.7 ms parallel	3.7 ms	fork 0.19–0.21 ms per input, but 12–13× slower than the sequential baseline	Not reproduced
Structured formats classified more accurately	98.7 % vs 93.2 %	0.979 vs 0.952 (per-label accuracy)	Qualitatively supported

The finding motivates this project's design: process creation per pattern (about 200 µs per input) dwarfs the matching itself (under a microsecond for five precompiled patterns), whereas splitting the *inputs* across cores scales, and one combined DFA is the fastest variant.

Demonstration

`make demo` runs a scripted walk-through on the sample logs: rule compilation and rejection of unsupported syntax, the NFA/DFA diagram, multi-label classification with checksum look-alikes left unflagged, masked PII, correlator escalations on the real OpenSSH log, identical output with 1 and 8 workers with the live statistics line, and the base-paper verdicts.

7. Remaining work

Next step	Content
Benchmark tooling (Phase 6)	Fork engine in the triage binary; --bench mode with latency histogram; synthetic generator with ground truth; rule sets of 5–64 rules; benchmark runner
Experiments (Phase 7)	Throughput and scaling of DFA vs POSIX vs fork engines over worker count, rule count and batch size; latency under load
Accuracy evaluation (Phase 7)	Full Loghub and AIT-LDS datasets; stratified sample labelled independently by both members; precision, recall, F1, Cohen's κ, confidence calibration
Optional extensions	DFA minimisation (Hopcroft's algorithm); ordered output; reproduction of the base paper's conflict-resolution scoring

References

[1] K. Thompson, “Programming Techniques: Regular expression search algorithm,” *Communications of the ACM*, 11(6), 1968.

[2] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2006 (ch. 3: lexical analysis, NFA/DFA construction).

[3] J. Zhu, S. He, P. He, J. Liu, M. R. Lyu, “Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics,” *IEEE ISSRE*, 2023.

[4] IEEE Std 1003.1 (POSIX), Regular Expressions; GNU C Library *regcomp/regexec*.

[5] Base paper: “Parallel Text Classification System Using Regex Pattern Matching on Multi-Core Architectures,” provided by the course faculty.

[6] fail2ban — intrusion prevention software (parameters *maxretry*, *findtime*).